Light

# Algorithms — Trace Tables & Dry Runs

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice


Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Trace table basics
2. Loops and conditions
3. Complex algorithms
Worked examples
Quick reference
Typical pitfalls


OCR GCSE Computer Science — **Algorithms: Trace Tables (J277/02 Section 2.1)**. This topic covers how to use trace tables to follow the execution of algorithms, recording variable values at each step, tracking loop iterations, and determining program outputs. Exam questions typically give you an algorithm and ask you to **complete a trace table**, **state the output**, or **identify errors** by tracing through the code. The 2024 exam included tracing a counting loop and determining the final values of multiple variables.

## What you must know

This checklist covers the key exam-visible skills. You should be able to perform each task accurately under exam conditions.

- **Set up** a trace table with columns for each variable and output.
- **Execute** code line by line, updating variable values in the table.
- **Trace** FOR loops, WHILE loops, and DO-UNTIL loops correctly.
- **Handle** IF-ELSE conditions, determining which branch executes.
- **Track** multiple variables that interact with each other.
- **Record** outputs (print statements) in the correct order.
- **State** the final values of variables after execution completes.
- **Identify** errors in algorithms by tracing unexpected results.

**Specification note.** OCR J277 Section 2.1 requires you to trace through algorithms using trace tables. You must be comfortable with the OCR Exam Reference Language (ERL) syntax. Questions typically award 1 mark per correctly traced row or for stating correct final values/outputs.

## Toolkit

### What is a trace table?

A **trace table** (also called a dry run) is a technique for manually working through an algorithm step by step, recording the values of variables as they change. It's like being the computer — you execute each instruction in order and write down what happens.

**Why use trace tables?**

- To **understand** what an algorithm does
- To **predict** the output of a program
- To **find errors** (bugs) in code
- To **verify** that an algorithm works correctly

### How to set up a trace table

**Step 1:** Create a column for each **variable** in the algorithm.
**Step 2:** Add an **Output** column if the algorithm prints anything.
**Step 3:** Optionally add a **Line** column to track which line you're on.
**Step 4:** Add a new row each time a variable **changes value**.

**Example structure:**

| Line | x | y | total | Output |
|---|---|---|---|---|
| 1 | 5 | | | |
| 2 | | 3 | | |
| 3 | | | 8 | |
| 4 | | | | 8 |

Only fill in cells where the value **changes**. Leave cells blank if that variable wasn't modified on that line.

### OCR Exam Reference Language (ERL) — key syntax

You need to recognise these constructs when tracing algorithms:

**Assignment:**

```
1  x = 5 2  y = x + 3 3  name = "Alice"
```

**FOR loop:**

```
for i = 1 to 5 print(i) next i
```

**WHILE loop:**

```
while x < 10 x = x + 1 endwhile
```

**IF-ELSE:**

```
if score >= 50 then print("Pass") else print("Fail") endif
```

### Tracing loops — counting iterations

**FOR loop rule:** The loop variable starts at the first value and increments by 1 each iteration until it exceeds the end value.

`for i = 1 to 5` means i takes values: 1, 2, 3, 4, 5 (5 iterations)

`for i = 0 to 3` means i takes values: 0, 1, 2, 3 (4 iterations)

**WHILE loop rule:** Check the condition BEFORE each iteration. If false, skip the loop body.

**DO-UNTIL loop rule:** Execute the body FIRST, then check the condition. Always runs at least once.

| Loop type | Condition checked | Minimum iterations |
|---|---|---|
| **FOR** | Before each iteration | 0 (if start > end) |
| **WHILE** | Before each iteration | 0 (if initially false) |
| **DO-UNTIL** | After each iteration | 1 (always runs once) |

### Common operators in ERL

| Operation | Symbol | Example | Result |
|---|---|---|---|
| Addition | + | 7 + 3 | 10 |
| Subtraction | - | 7 - 3 | 4 |
| Multiplication | * | 7 * 3 | 21 |

| Operation | Symbol | Example | Result |
|---|---|---|---|
| Division | / | 7 / 2 | 3.5 |
| Integer division | DIV | 7 DIV 2 | 3 |
| Modulus (remainder) | MOD | 7 MOD 2 | 1 |
| Exponentiation | ^ | 2 ^ 3 | 8 |

**Exam tip — DIV and MOD.**
**DIV** gives the whole number of times one number fits into another (7 DIV 2 = 3).
**MOD** gives the remainder (7 MOD 2 = 1).
These are commonly tested. Remember: 7 = (2 × 3) + 1, so DIV gives 3, MOD gives 1.

## 1. Trace table basics

A trace table allows you to work through an algorithm systematically, keeping track of what each variable contains at each step. The key is to **be the computer** — execute each line in order, exactly as written, without skipping ahead or assuming you know what will happen.

### The golden rules

1. **Execute line by line** — never skip steps
2. **Update only when changed** — add a new row when a variable gets a new value
3. **Use current values** — when calculating, use the most recent value of each variable
4. **Don't predict** — work through methodically, don't guess the answer
5. **Check your arithmetic** — silly mistakes lose marks

### Reading the current value

When a variable is used in a calculation, look UP the column to find its most recent value. The current value is the last value written in that column.

| x | y |
|---|---|
| 5 | |
| | 10 |
| 8 | |
| | ? |

If the next line is `y = x + 2`, we use x = 8 (the most recent), so y = 10.

### Simple sequential algorithms

Let's trace through a simple algorithm with no loops or conditions:

```
1  a = 10 2  b = 3 3  c = a + b 4  a = a - 5 5  print(a) 6  print(c)
```

| Line | a | b | c | Output |
|---|---|---|---|---|
| 1 | 10 | | | |
| 2 | | 3 | | |
| 3 | | | 13 | |
| 4 | 5 | | | |
| 5 | | | | 5 |
| 6 | | | | 13 |

Notice how on line 4, we calculate `a - 5` using the current value of a (which is 10), giving 5. The variable `c` doesn't change — it still holds 13 from when it was calculated on line 3.

**Exam tip — variables don't update automatically.**
A common misconception: when `a` changes on line 4, students think `c` should also change. It doesn't! `c` was set to 13 on line 3 and stays at 13 unless explicitly changed.

Worked Example 1 – Simple sequential trace

**Question:** Trace the following algorithm and state the output. [3 marks]

```
1  x = 8 2  y = 2 3  z = x * y 4  x = z - x 5  print(x)
```

**Step 1:** Line 1 — x = 8
**Step 2:** Line 2 — y = 2
**Step 3:** Line 3 — z = 8 * 2 = 16
**Step 4:** Line 4 — x = 16 - 8 = 8
**Step 5:** Line 5 — print(x) outputs 8

| x | y | z | Output |
|---|---|---|---|
| 8 | | | |
| | 2 | | |
| | | 16 | |
| 8 | | | |
| | | | 8 |

**Answer:** Output is **8**

**Exam tip:** On line 4, we use the current values: z = 16 and x = 8. The new value of x (8) happens to be the same as before, but that's just coincidence!

### Worked Example 2 — Using DIV and MOD

**Question:** Trace the following algorithm and state the final values of q and r. [3 marks]

```
1  n = 17 2  d = 5 3  q = n DIV d 4  r = n MOD d 5  print(q, r)
```

**Step 1:** Line 1 — n = 17
**Step 2:** Line 2 — d = 5
**Step 3:** Line 3 — q = 17 DIV 5 = 3 (how many times does 5 go into 17? 3 times)
**Step 4:** Line 4 — r = 17 MOD 5 = 2 (what's the remainder? 17 - 15 = 2)

| n | d | q | r | Output |
|---|---|---|---|---|
| 17 | | | | |
| | 5 | | | |
| | | 3 | | |
| | | | 2 | |
| | | | | 3 2 |

**Answer:** q = **3**, r = **2**

**Exam tip:** Check DIV and MOD: 17 = (5 × 3) + 2, so DIV = 3 and MOD = 2. They should reconstruct the original: (d × q) + r = n.

### Worked Example 3 — String concatenation

**Question:** Trace the following algorithm and state the output. [2 marks]

```
1  first = "Hello" 2  second = "World" 3  message = first + " "
+ second 4  print(message)
```

**Step 1:** Line 1 — first = "Hello"

**Step 2:** Line 2 — second = "World"
**Step 3:** Line 3 — message = "Hello" + " " + "World" = "Hello World"
**Step 4:** Line 4 — print outputs "Hello World"

| first | second | message | Output |
|---|---|---|---|
| "Hello" | | | |
| | "World" | | |
| | | "Hello World" | |
| | | | Hello World |

**Answer:** Output is **Hello World**

**Exam tip:** String concatenation uses + to join strings. Don't forget any spaces — "Hello" + "World" = "HelloWorld" (no space)!

Worked Example 4 – Simple IF statement

**Question:** Trace the following algorithm when the user enters 75. State the output. [3 marks]

```
1  score = input("Enter score: ") 2  if score >= 50 then 3
grade = "Pass" 4  else 5   grade = "Fail" 6  endif
7  print(grade)
```

**Step 1:** Line 1 — score = 75 (user input)
**Step 2:** Line 2 — Check: is 75 >= 50? YES, so execute line 3
**Step 3:** Line 3 — grade = "Pass"
**Step 4:** Skip lines 4-5 (else branch not taken), go to line 7
**Step 5:** Line 7 — print "Pass"

| score | grade | Output |
|---|---|---|
| 75 | | |
| | "Pass" | |
| | | Pass |

**Answer:** Output is **Pass**

> **Exam tip:** When the IF condition is true, the ELSE branch is skipped entirely. Only one branch executes.

## 2. Loops and conditions

Most trace table questions involve loops because they test your ability to track variables through multiple iterations. The key is to be systematic — add a new row for each time through the loop and carefully check the loop condition.

### Tracing FOR loops

A FOR loop has a loop variable that automatically increments. You need to track this variable and any others modified inside the loop.

```
for i = 1 to 4 total = total + i next i
```

The loop runs with i = 1, 2, 3, 4 (four iterations). After i = 4 completes, the loop ends.

> **Exam tip — FOR loop count.**
> `for i = 1 to n` runs **n times**.
> `for i = 0 to n` runs **n+1 times**.
> `for i = 1 to n-1` runs **n-1 times**.

### Tracing WHILE loops

A WHILE loop checks its condition before each iteration. If the condition is false initially, the loop body never executes.

```
x = 1 while x < 5 print(x) x = x + 1 endwhile
```

Check: is 1 < 5? Yes → print 1, x becomes 2.
Check: is 2 < 5? Yes → print 2, x becomes 3.
Check: is 3 < 5? Yes → print 3, x becomes 4.
Check: is 4 < 5? Yes → print 4, x becomes 5.
Check: is 5 < 5? No → loop ends.
Output: 1 2 3 4

### Accumulator patterns

A common pattern is using a variable to accumulate a total or count. Make sure to initialise the accumulator before the loop!

**Running total:**

```
total = 0 for i = 1 to 4 total = total + i next i print(total)
```

Output: 10 (because 1+2+3+4 = 10)

**Counter:**

```
count = 0 for i = 1 to 5 if i MOD 2 == 0 then count = count + 1
endif next i print(count)
```

Output: 2 (counts even numbers: 2 and 4)

**Common pattern — sum of 1 to n.**
The sum 1+2+3+...+n can be traced with a loop or calculated directly as n(n+1)/2. For n=4: 4×5÷2 = 10. This is useful for checking your trace table answer.

### Worked Example 5 — FOR loop with accumulator [2024 exam style]

**Question:** Complete the trace table for the following algorithm and state the final output. [4 marks]

```
1  total = 0 2  for count = 1 to 3 3   total = total + count * 2
4  next count 5  print(total)
```

**Initial:** total = 0
**Iteration 1:** count = 1, total = 0 + (1 × 2) = 2
**Iteration 2:** count = 2, total = 2 + (2 × 2) = 6
**Iteration 3:** count = 3, total = 6 + (3 × 2) = 12
**Loop ends, print total**

| count | total | Output |
|---|---|---|
| | 0 | |
| 1 | 2 | |
| 2 | 6 | |
| 3 | 12 | |
| | | 12 |

**Answer:** Output is **12**

**Exam tip:** Show each iteration clearly. The loop runs 3 times (count = 1, 2, 3), so you should have 3 rows of changes plus the initial value and final output.

### Worked Example 6 — WHILE loop trace

**Question:** Trace the following algorithm. State the output and the final value of x. [4 marks]

```
x = 10
while x > 3
  print(x)
  x = x - 3
endwhile
```

**Initial:** x = 10
**Check:** 10 > 3? Yes → print 10, x = 10 - 3 = 7
**Check:** 7 > 3? Yes → print 7, x = 7 - 3 = 4
**Check:** 4 > 3? Yes → print 4, x = 4 - 3 = 1
**Check:** 1 > 3? No → loop ends

| x | Output |
|---|---|
| 10 | |
| | 10 |
| 7 | |
| | 7 |
| 4 | |
| | 4 |
| 1 | |

**Answer:** Output is **10, 7, 4**. Final value of x is **1**.

**Exam tip:** The final value of x is 1, not 4! The loop continues until x becomes 1, then the condition (1 > 3) is false and the loop stops.

Worked Example 7 — Nested IF in a loop

**Question:** Trace the algorithm and state the final value of result. [4 marks]

```
result = 0
for n = 1 to 5
  if n MOD 2 == 1 then
result = result + n
  endif
next n
```

**What does n MOD 2 == 1 test?** Whether n is odd (odd numbers give remainder 1 when divided by 2).
**Iteration 1:** n=1, 1 MOD 2 = 1, 1==1? Yes → result = 0 + 1 = 1
**Iteration 2:** n=2, 2 MOD 2 = 0, 0==1? No → result unchanged (1)
**Iteration 3:** n=3, 3 MOD 2 = 1, 1==1? Yes → result = 1 + 3 = 4
**Iteration 4:** n=4, 4 MOD 2 = 0, 0==1? No → result unchanged (4)
**Iteration 5:** n=5, 5 MOD 2 = 1, 1==1? Yes → result = 4 + 5 = 9

| n | n MOD 2 | result |
|---|---|---|
| | | 0 |
| 1 | 1 | 1 |
| 2 | 0 | |
| 3 | 1 | 4 |
| 4 | 0 | |
| 5 | 1 | 9 |

**Answer:** Final value of result is **9** (sum of odd numbers 1+3+5)

**Exam tip:** Adding an intermediate column (like n MOD 2) can help you keep track of condition evaluations. It's not always required but can prevent errors.

### Worked Example 8 — DO-UNTIL loop

**Question:** Trace the algorithm and state how many times the loop body executes. [3 marks]

```
1  x = 1 2  do 3   x = x * 2 4  until x > 10 5  print(x)
```

**Initial:** x = 1
**Execute body:** x = 1 * 2 = 2. Check: 2 > 10? No → continue
**Execute body:** x = 2 * 2 = 4. Check: 4 > 10? No → continue
**Execute body:** x = 4 * 2 = 8. Check: 8 > 10? No → continue
**Execute body:** x = 8 * 2 = 16. Check: 16 > 10? Yes → loop ends

| Iteration | x (before) | x (after) | x > 10? |
|---|---|---|---|
| 1 | 1 | 2 | No |
| 2 | 2 | 4 | No |
| 3 | 4 | 8 | No |
| 4 | 8 | 16 | Yes |

**Answer:** Loop executes **4 times**. Output is **16**.

> **Exam tip:** DO-UNTIL executes the body FIRST, then checks the condition. The loop stops when the condition becomes TRUE (opposite to WHILE).

## 3. Complex algorithms

Higher-difficulty trace table questions involve multiple interacting variables, nested loops, arrays, or algorithms where you need to identify errors. The same principles apply — be systematic and don't skip steps.

### Tracing with arrays

When tracing algorithms with arrays, track the array elements that change. You may need to show the entire array or just specific elements.

```
nums = [3, 1, 4, 1, 5] total = 0 for i = 0 to 4 total = total +
nums[i] next i
```

Here, the array doesn't change — only `total` and `i` change. Track those.

### Swapping variables

A common algorithm pattern is swapping two values using a temporary variable:

```
a = 5 b = 9 temp = a // temp = 5 a = b // a = 9 b = temp // b = 5
```

Without `temp`, one value would be lost. Trace this carefully — the order matters!

### Finding errors using trace tables

Sometimes you'll be asked to trace an algorithm to find a bug. The trace table reveals what actually happens versus what was intended.

**Buggy code — counting down:**

```
x = 5 while x > 0 print(x) x = x - 1 endwhile print("Done")
```

This prints: 5, 4, 3, 2, 1, Done

If we wanted to include 0, we'd need `x >= 0`.

**Off-by-one error example:**

```
// Intended: sum 1 to 5 total = 0 for i = 1 to 4 // Bug! total =
total + i next i
```

This gives 10 (1+2+3+4) instead of 15 (1+2+3+4+5).

The loop should be `for i = 1 to 5`.

### Linear search trace

A classic algorithm to trace is linear search — looking through an array for a target value:

```
items = ["cat", "dog", "bird", "fish"] target = "bird" found = false
for i = 0 to 3 if items[i] == target then found = true position = i
endif next i
```

The loop checks each element. When i=2, items[2]="bird" matches target, so found becomes true and position becomes 2.

### Worked Example 9 — Array traversal

**Question:** Trace the algorithm and state the final output. [4 marks]

```
1  data = [2, 5, 3, 8] 2  max = data[0] 3  for i = 1 to 3 4   if
data[i] > max then 5   max = data[i] 6   endif 7  next i
8  print(max)
```

**What does this algorithm do?** Finds the maximum value in the array.
**Initial:** max = data[0] = 2
**i=1:** data[1]=5, 5 > 2? Yes → max = 5
**i=2:** data[2]=3, 3 > 5? No → max unchanged
**i=3:** data[3]=8, 8 > 5? Yes → max = 8

| i | data[i] | data[i] > max? | max |
|---|---|---|---|
| | | | 2 |
| 1 | 5 | Yes | 5 |
| 2 | 3 | No | |
| 3 | 8 | Yes | 8 |

**Answer:** Output is **8**

**Exam tip:** This is the "find maximum" algorithm. Notice the loop starts at index 1, not 0, because max is initialised to data[0].

### Worked Example 10 — Variable swap trace

**Question:** Trace the algorithm and state the final values of a and b. [3 marks]

```
1  a = 15 2  b = 27 3  temp = a 4  a = b 5  b = temp 6  print(a,
b)
```

**Step 1:** Line 1 — a = 15
**Step 2:** Line 2 — b = 27

**Step 3:** Line 3 — temp = 15 (copy a's value before it changes)
**Step 4:** Line 4 — a = 27 (a now has b's value)
**Step 5:** Line 5 — b = 15 (b now has original a's value via temp)

| Line | a | b | temp |
|---|---|---|---|
| 1 | 15 | | |
| 2 | | 27 | |
| 3 | | | 15 |
| 4 | 27 | | |
| 5 | | 15 | |

**Answer:** Final values: a = **27**, b = **15** (swapped)

**Exam tip:** The temp variable is essential. Without it, one value would be overwritten before it could be copied.

Worked Example 11 — Identifying an error

**Question:** The following algorithm is intended to calculate the average of 5 numbers input by the user. Trace it with inputs 10, 20, 30, 40, 50 and identify the error. [5 marks]

```
1  total = 0 2  for i = 1 to 4 3   num = input("Enter number: ")
4   total = total + num 5  next i 6  average = total / 5
7  print(average)
```

**Trace with inputs 10, 20, 30, 40, 50:**

| i | num | total | average |
|---|---|---|---|
| | | 0 | |
| 1 | 10 | 10 | |
| 2 | 20 | 30 | |
| 3 | 30 | 60 | |
| 4 | 40 | 100 | |
| | | | 20 |

**Expected:** (10+20+30+40+50)/5 = 150/5 = 30
**Actual:** (10+20+30+40)/5 = 100/5 = 20

**Error:** The loop only runs 4 times (i = 1 to 4), but we need to input 5 numbers.
**Fix:** Change line 2 to `for i = 1 to 5`

**Exam tip:** This is an "off-by-one" error. The loop runs 4 times but divides by 5. Always check that loop bounds match the intended count.

### Worked Example 12 — Nested loop trace

**Question:** Trace the algorithm and state all outputs in order. [5 marks]

```
1  for i = 1 to 2 2   for j = 1 to 3 3   print(i, j) 4   next j
5  next i
```

**Outer loop i=1:**

- Inner loop j=1: print(1, 1) → outputs "1 1"
- Inner loop j=2: print(1, 2) → outputs "1 2"
- Inner loop j=3: print(1, 3) → outputs "1 3"

**Outer loop i=2:**

- Inner loop j=1: print(2, 1) → outputs "2 1"
- Inner loop j=2: print(2, 2) → outputs "2 2"
- Inner loop j=3: print(2, 3) → outputs "2 3"

| i | j | Output |
|---|---|---|
| 1 | 1 | 1 1 |
| | 2 | 1 2 |
| | 3 | 1 3 |
| 2 | 1 | 2 1 |
| | 2 | 2 2 |
| | 3 | 2 3 |

**Answer:** Outputs are: **1 1, 1 2, 1 3, 2 1, 2 2, 2 3**

**Exam tip:** For nested loops, the inner loop completes ALL its iterations for EACH iteration of the outer loop. Total iterations = outer × inner (here: 2 × 3 = 6).

## Summary of worked examples

| # | Topic | Algorithm type | Key skill |
|---|---|---|---|
| 1 | Sequential trace | Simple assignments | Using current values in calculations |
| 2 | DIV and MOD | Integer operations | Calculating quotient and remainder |
| 3 | String concatenation | String operations | Joining strings with + |
| 4 | IF statement | Selection | Determining which branch executes |
| 5 | FOR loop (2024 style) | Count-controlled loop | Accumulator pattern |
| 6 | WHILE loop | Condition-controlled loop | Pre-condition checking |
| 7 | IF inside loop | Selection + iteration | Counting with conditions |
| 8 | DO-UNTIL loop | Post-condition loop | Loop runs at least once |
| 9 | Array traversal | Find maximum | Accessing array elements |
| 10 | Variable swap | Using temp variable | Order of operations matters |
| 11 | Finding errors | Off-by-one bug | Tracing to identify bugs |
| 12 | Nested loops | Loop inside loop | Inner loop completes for each outer |

## Quick reference

### Key terms

| Term | Definition (exam-answer style) |
|---|---|
| **Trace table** | A technique for manually executing an algorithm step by step, recording variable values as they change |
| **Dry run** | Another name for tracing through code without actually running it on a computer |
| **Iteration** | One complete execution of a loop body |
| **Accumulator** | A variable that accumulates a running total or count through iterations |
| **Loop counter** | A variable that controls how many times a loop executes (e.g., i in a FOR loop) |

| Term | Definition (exam-answer style) |
|---|---|
| **Off-by-one error** | A bug where a loop runs one too many or one too few times |
| **DIV** | Integer division — gives the whole number result (17 DIV 5 = 3) |
| **MOD** | Modulus — gives the remainder after division (17 MOD 5 = 2) |

## Loop comparison

| Loop type | When to use | Condition check | Minimum runs |
|---|---|---|---|
| **FOR** | Known number of iterations | Before each iteration | 0 |
| **WHILE** | Unknown iterations, may not run at all | Before each iteration | 0 |
| **DO-UNTIL** | Must run at least once | After each iteration | 1 |

## Common operators

| Operator | Meaning | Example | Result |
|---|---|---|---|
| + | Add | 5 + 3 | 8 |
| - | Subtract | 5 - 3 | 2 |
| * | Multiply | 5 * 3 | 15 |
| / | Divide | 5 / 2 | 2.5 |
| DIV | Integer divide | 5 DIV 2 | 2 |
| MOD | Remainder | 5 MOD 2 | 1 |
| ^ | Power | 2 ^ 3 | 8 |
| == | Equal to | 5 == 5 | true |
| != | Not equal | 5 != 3 | true |
| < | Less than | 3 < 5 | true |
| > | Greater than | 5 > 3 | true |
| <= | Less than or equal | 5 <= 5 | true |

| Operator | Meaning | Example | Result |
|---|---|---|---|
| >= | Greater than or equal | 5 >= 3 | true |

## Typical pitfalls

**Pitfall 1 — Using old values.**
**Wrong:** Using a variable's initial value instead of its current value.
**Right:** Always look UP the column to find the **most recent value** of a variable.

**Pitfall 2 — Variables updating automatically.**
**Wrong:** Thinking c changes when a changes after `c = a + b`.
**Right:** Variables only change when they appear on the **left side** of an assignment. c stays the same until a new `c = ...` line executes.

**Pitfall 3 — Wrong loop count.**
**Wrong:** `for i = 1 to 5` runs 4 times.
**Right:** It runs **5 times** (i = 1, 2, 3, 4, 5). The end value IS included.

**Pitfall 4 — WHILE vs DO-UNTIL condition.**
**Wrong:** WHILE loops run until the condition is true.
**Right:** WHILE loops run **while** condition is true. DO-UNTIL runs **until** condition is true.

**Pitfall 5 — Forgetting to initialise accumulators.**
**Wrong:** Starting a loop without setting `total = 0` first.
**Right:** Always initialise counters and accumulators **before** the loop.

**Pitfall 6 — DIV vs / (normal division).**
**Wrong:** 7 DIV 2 = 3.5
**Right:** 7 DIV 2 = **3** (integer part only). 7 / 2 = 3.5 (full division).

**Pitfall 7 — Array index errors.**
**Wrong:** First element is data[1].
**Right:** In most languages (including ERL), arrays start at index **0**. First element is data[0].

**Pitfall 8 — Skipping the final iteration.**
**Wrong:** Not tracing the last time through a loop.
**Right:** Trace **every** iteration, including the last one. Then check if the loop condition allows another iteration.

**Pitfall 9 — Arithmetic errors.**
**Wrong:** Rushing calculations and making basic arithmetic mistakes.
**Right:** Take your time. Double-check calculations, especially in loops where errors compound.

**Pitfall 10 — Confusing = and ==.**
**Wrong:** Thinking `x = 5` checks if x equals 5.
**Right:** `=` is assignment (sets value). `==` is comparison (checks equality).

Assessment Hub | Long-Answer | Notes Index

**Williams Physics Education** — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.